PROJECT REPORT

1. Project Title

Music Store Management

2. Student Details

Student Name: Jitanshu Borana
Enrollment Number:IU2541230538 , 3CSE - E2
Project Code:CPP63
Institute:IITE
Program: Btech

3. Problem Statement

Managing a music store manually can be difficult because album details, artist information, customer records, stock availability, and sales transactions must be maintained accurately. Manual record-keeping can lead to errors, difficulty finding album information, incorrect stock tracking, and loss of records over time.

The **Music Store Management System** addresses these problems by providing a menu-driven C++ application that allows users to add, display, and search albums, manage artist and customer details, process sales, and update album stock. The system also uses file handling to save records in text files and load them when the program starts, helping maintain data between sessions and making music store operations easier to manage.

4. Objectives

1. To develop a menu-driven Music Store Management System using C++.
2. To maintain album details, including album ID, title, artist ID, genre, price, and available stock.
3. To manage artist and customer records efficiently.
4. To provide functionality for adding, displaying, and searching albums.
5. To process sales transactions and calculate the total sale amount.
6. To update album stock automatically after a sale.
7. To implement Object-Oriented Programming (OOP) concepts such as classes, objects, encapsulation, inheritance, and polymorphism.
8. To use file handling to store and retrieve records from text files.
9. To maintain data between program executions by loading saved records when the program starts.
10. To reduce manual record-keeping errors and make music store operations more organized and efficient.

5. Features

The Music Store Management System is a menu-driven C++ application that helps manage albums, artists, customers, and sales. It provides the following features:

1. **Add Album:** Allows the user to enter album details, including album ID, title, artist ID, genre, price, and stock quantity.
2. **Display All Albums:** Displays the details of all albums available in the music store.
3. **Search Album:** Allows the user to search for an album using its ID and view its details if found.
4. **Add Artist:** Allows the user to add artist information, including artist ID, name, and genre.
5. **Display Artists:** Displays the details of all registered artists.
6. **Add Customer:** Allows the user to register a customer by entering customer details such as customer ID, name, and phone number.
7. **Display Customers:** Displays the details of all registered customers.
8. **Process Sale:** Processes an album sale by accepting the customer ID, album ID, and quantity. It calculates the total amount, records the sale, and updates the available stock.
9. **Exit:** Saves the records to text files and terminates the program.

File Handling: The system uses text files to store album, artist, customer, and sales records. Saved data is loaded when the program starts, helping preserve records between sessions.

These features make the system easier to operate and help organize the daily activities of a music store.

6. Classes and Responsibilities

1. Record Class

- **Data Members:** id
- **Functions:** getId(), display(), constructors, destructor.
- **Responsibility:** Abstract base class for common record properties and behavior.

2. Album Class

- **Data Members:** title, artistId, genre, price, stock
- **Functions:** display(), updateStock(), getPrice(), getStock(), sellStock().
- **Responsibility:** Manages album details and stock.

3. Artist Class

- **Data Members:** name, genre
- **Functions:** display()
- **Responsibility:** Stores and displays artist information.

4. Customer Class

- **Data Members:** name, phone
- **Functions:** display()
- **Responsibility:** Stores and displays customer information.

5. Sale Class

- **Data Members:** customerId, albumId, quantity, totalAmount
- **Functions:** display()
- **Responsibility:** Stores and displays sales transaction details.

6. FileManager Class

- **Data Members:** No major instance data members; uses file-handling functions.
- **Functions:** saveAlbums(), loadAlbums(), saveArtists(), loadArtists(), saveCustomers(), loadCustomers(), saveSales(), loadSales().
- **Responsibility:** Saves and loads records using text files.

7. MusicStore Class

- **Data Members:** albums, artists, customers, sales (vectors).
- **Functions:** addAlbum(), displayAlbums(), searchAlbum(), addArtist(), addCustomer(), processSale(), saveData(), loadData().
- **Responsibility:** Manages store operations and coordinates records and sales.

8. Main Function

- **Responsibility:** Displays the menu, accepts user choices, calls appropriate functions, and saves data before exiting.

7. OOP Concepts Used

1. Classes and Objects: Classes define the structure of entities, while objects are their instances. For example, Album is a class, and an album created with its details is an object.

2. Encapsulation: Data members are kept private or protected and accessed through public member functions. For example, Album stores price and stock privately and provides functions such as getPrice() and getStock().

3. Constructors and Destructor: Constructors initialize objects, such as Record(int recordId), which initializes the record ID. The virtual destructor ~Record() supports safe destruction through a base-class pointer.

4. Inheritance: The Album, Artist, Customer, and Sale classes inherit from the Record class, reusing common properties such as id and the getId() function.

5. Runtime Polymorphism: The display() function is declared as a pure virtual function in Record and implemented in derived classes. Calling display() through a base-class pointer can invoke the appropriate derived-class version at runtime.

6. Dynamic Memory: Dynamic memory is allocated at runtime using new and released using delete. The project uses vectors to manage collections of records, which handle their own memory automatically; explicit dynamic allocation should only be mentioned if new or equivalent allocation is actually used in the code.

8. File Handling

1. Data Stored: The system stores album details, artist information, customer records, and sales transactions, including the quantity sold and total sale amount.

2. File Format Used: The project uses plain text files (.txt) to store records. Fields are separated by the pipe symbol (|). The files used are albums.txt, artists.txt, customers.txt, and sales.txt.

3. Saving and Loading Process: The FileManager class provides functions to save records to text files and load them when required. When the program starts, saved records are loaded into the system. When records are modified or the program exits, the data is saved to the corresponding files.

4. Persistence Testing: The system was tested by adding records and processing a sale, then closing and restarting the program. The saved records were loaded again, and the updated album stock and sales information remained available. This confirmed that data persists between program executions.

9. UML Diagrams

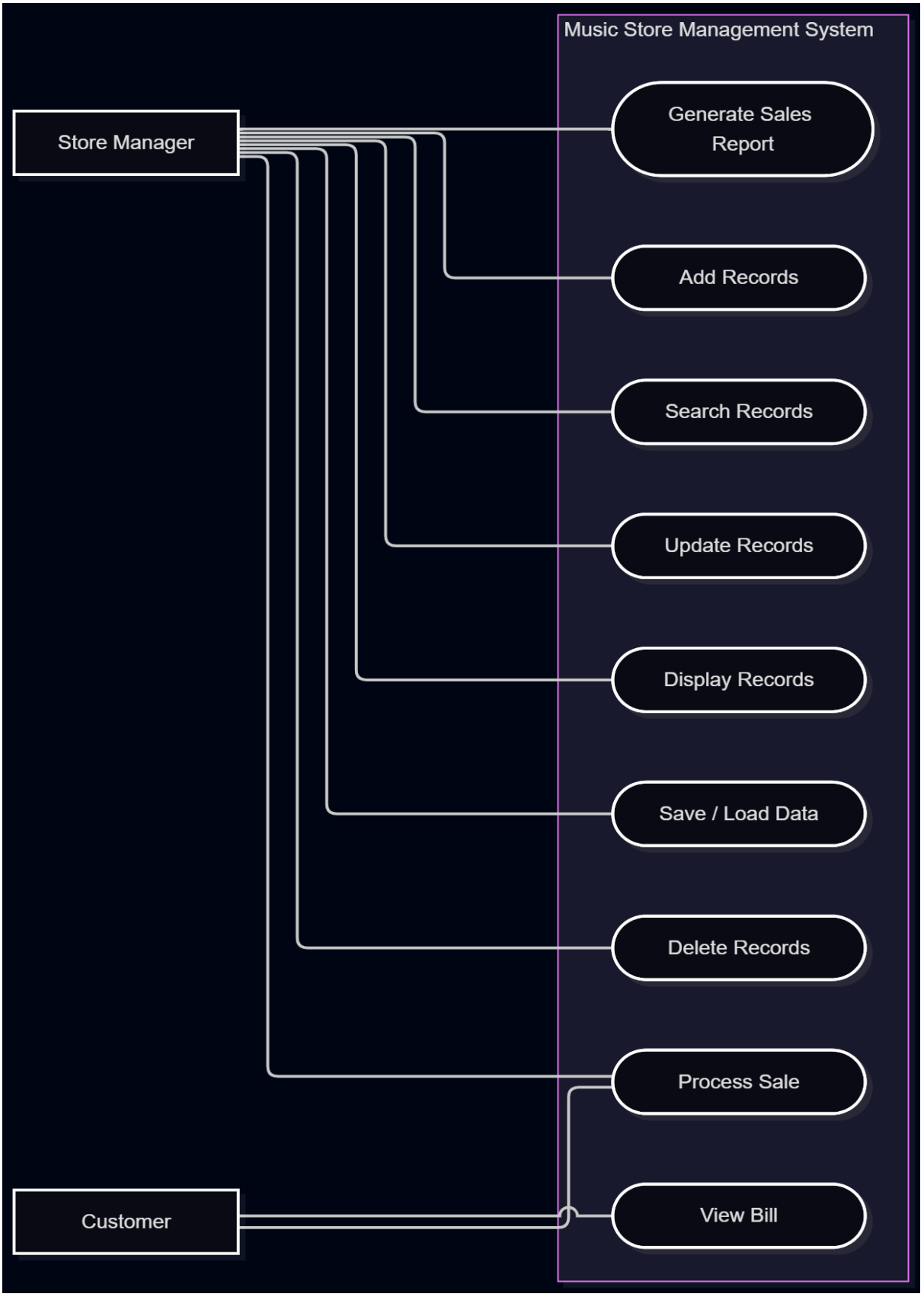


Figure 1: Use Case Diagram

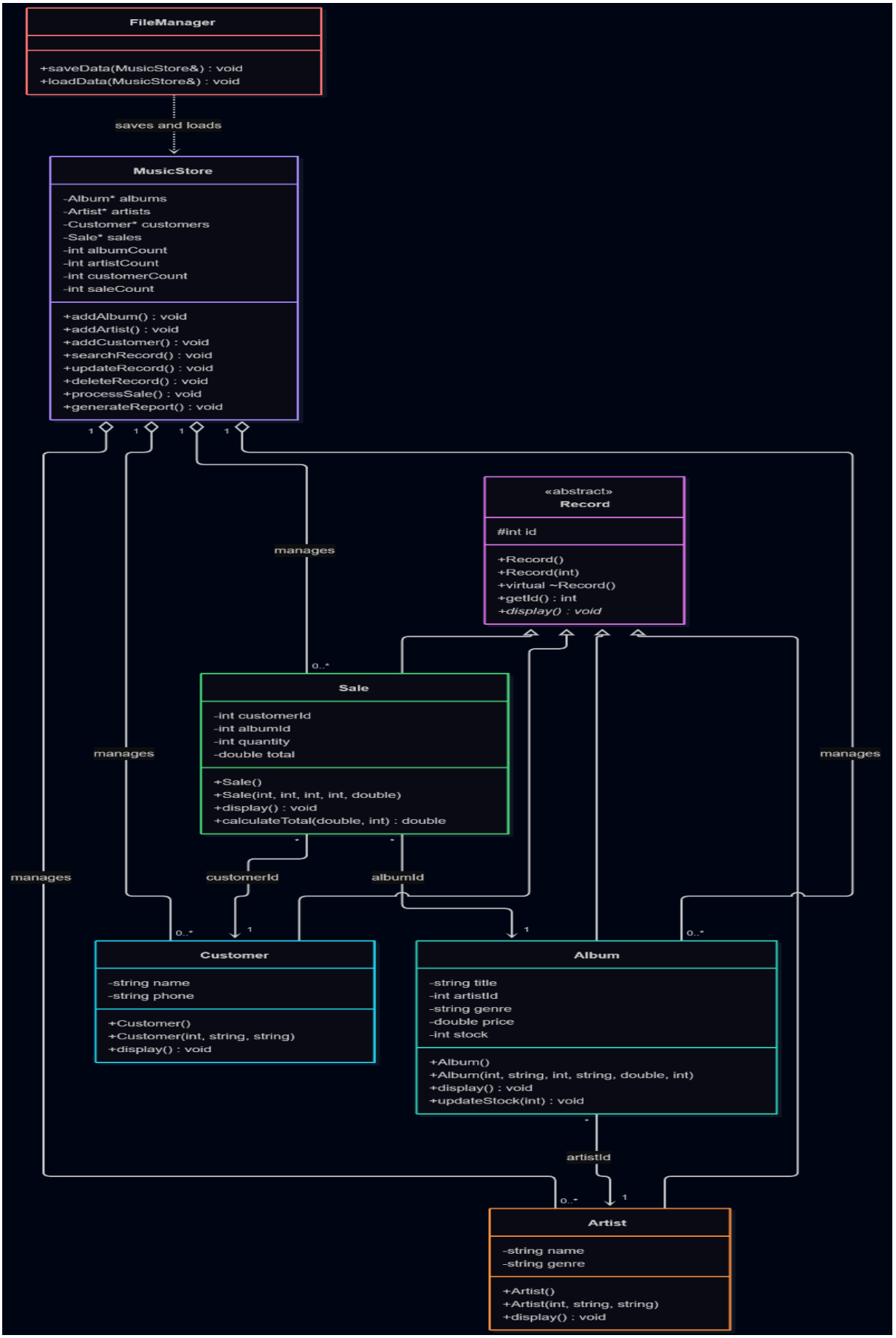


Figure 2: Class Diagram

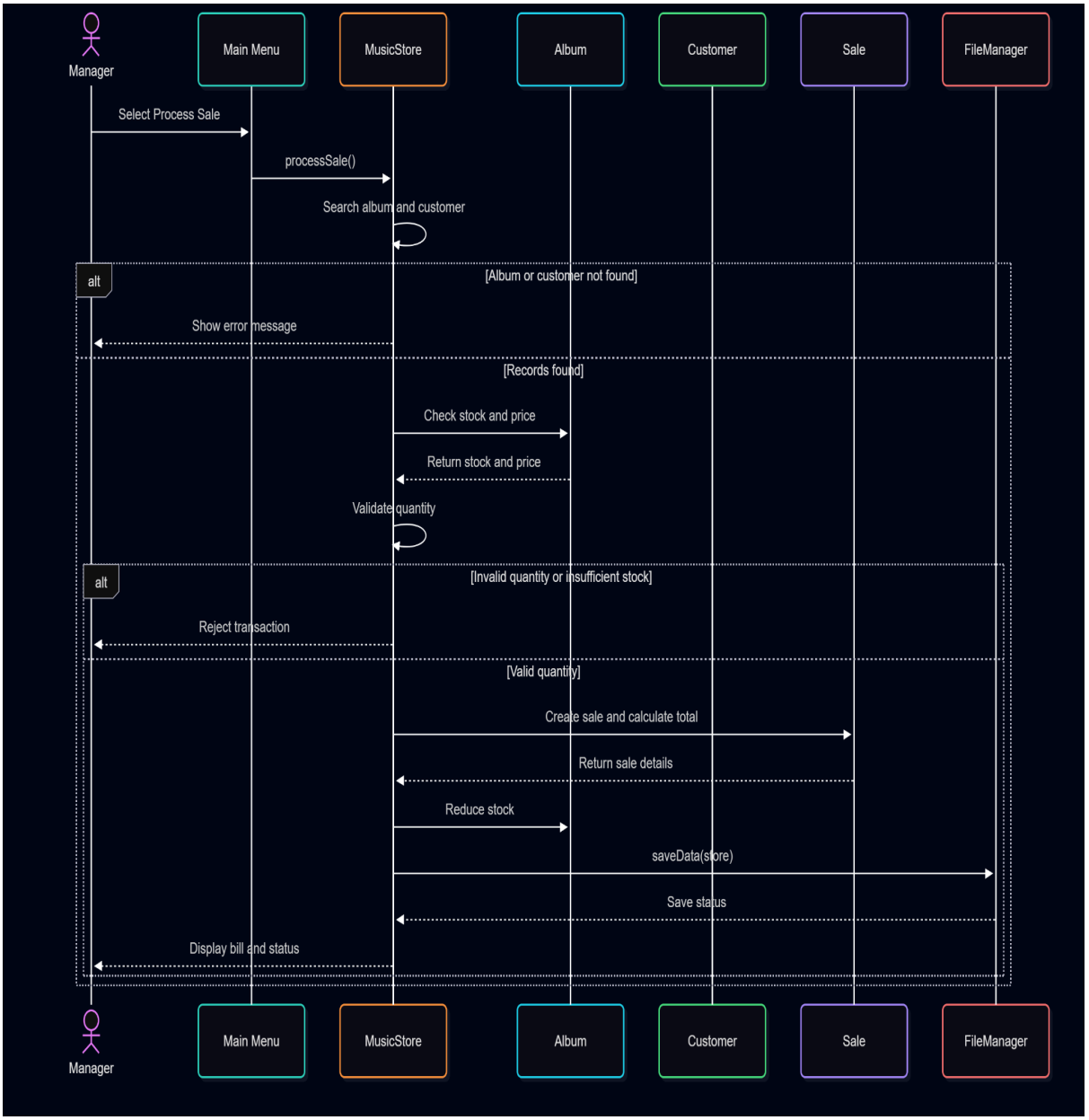


Figure 3:Sequence diagram 1

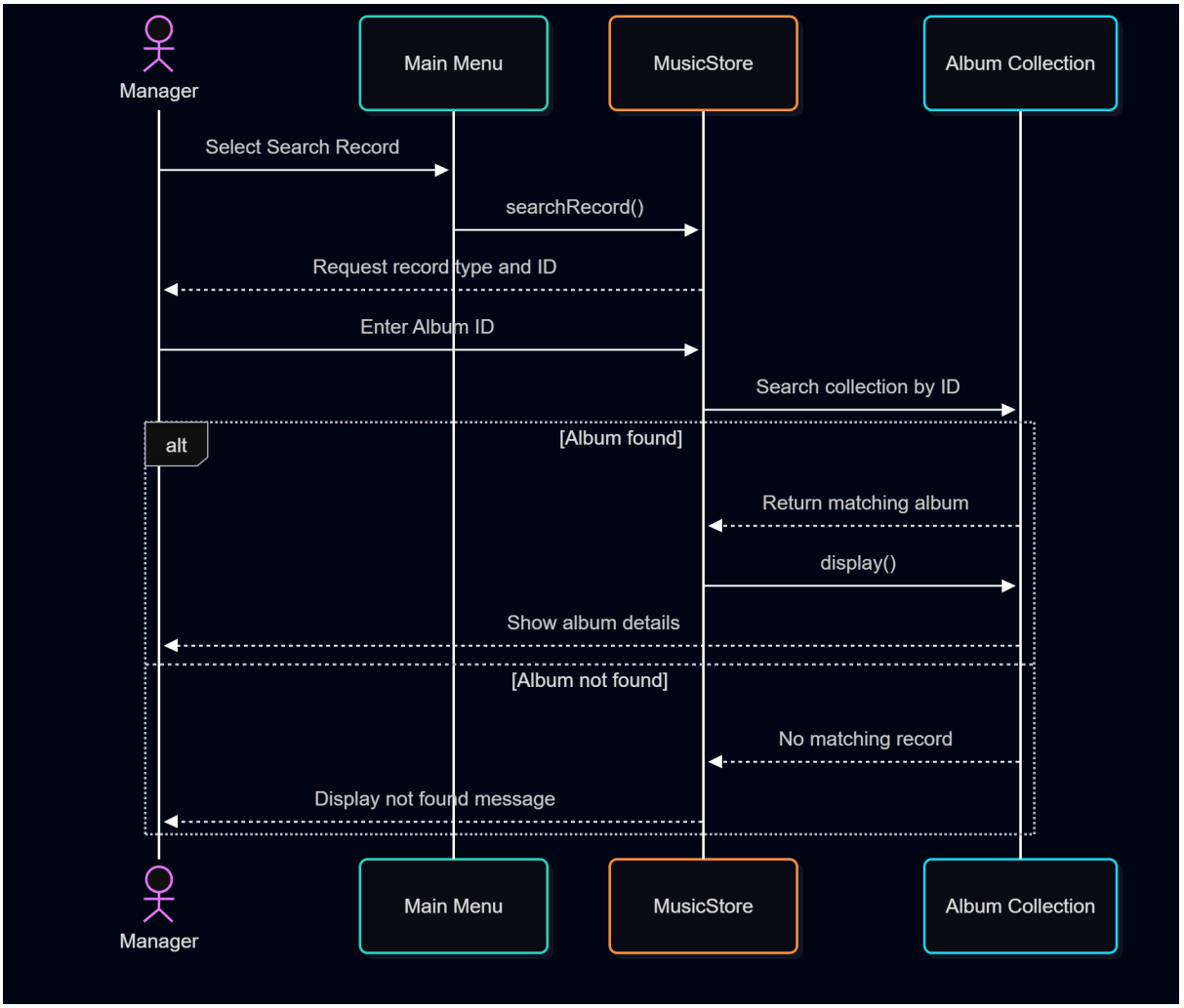


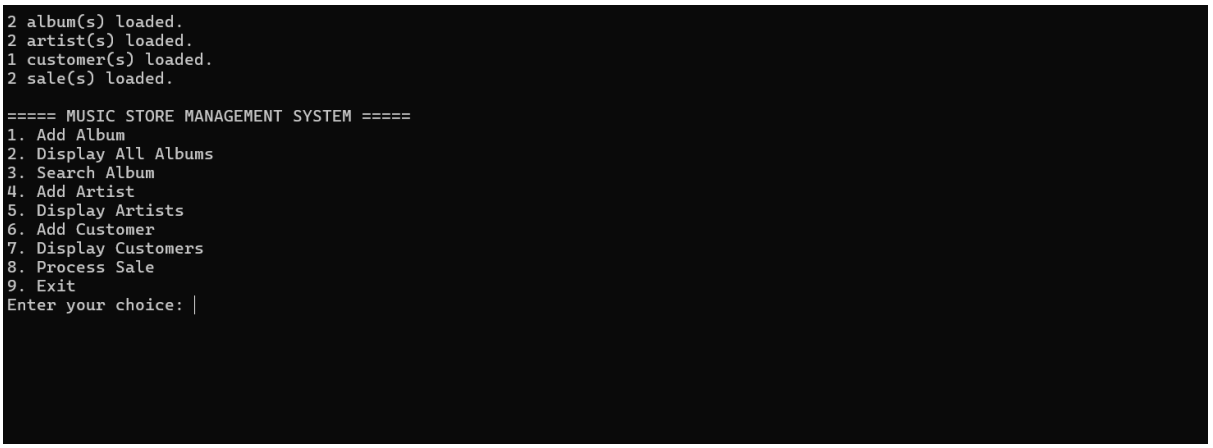
Figure 4: Sequence diagram 2

10. Program Workflow

1. **Program Start:** The main() function starts execution and creates a MusicStore object.
2. **Load Data:** Previously saved album, artist, customer, and sales records are loaded from text files.
3. **Display Menu:** The system displays the main menu with nine options.
4. **Accept User Choice:** The user selects an operation, such as adding an album, displaying records, or searching for an album.
5. **Perform Operation:** The corresponding MusicStore functions execute the selected operation.
6. **Process Sales:** When a sale is selected, the system records the transaction, calculates the total amount, and updates the album stock.
7. **Repeat Operations:** The menu is displayed again until the user selects Exit.
8. **Save Data:** The system saves the records to text files to preserve changes.
9. **Program Exit:** The application terminates after saving the data.

This workflow provides organized store management and maintains records between program executions.

11. Screenshots







```
Enter your choice: 7

--- Customer Details ---
Customer ID: 700
Name: Rahul
Phone: 9876543210

--- Customer Details ---
Customer ID: 158
Name: Jay Shah
Phone: 3578945688

===== MUSIC STORE MANAGEMENT SYSTEM =====
1. Add Album
2. Display All Albums
3. Search Album
4. Add Artist
5. Display Artists
6. Add Customer
7. Display Customers
8. Process Sale
9. Exit
Enter your choice:
```

```
===== MUSIC STORE MANAGEMENT SYSTEM =====
1. Add Album
2. Display All Albums
3. Search Album
4. Add Artist
5. Display Artists
6. Add Customer
7. Display Customers
8. Process Sale
9. Exit
Enter your choice: 2

--- Album Details ---
Album ID: 111
Title: let down
Artist ID: 544
Genre: emotional
Price: 533
Stock: 20

--- Album Details ---
Album ID: 756
Title: Thunder
Artist ID: 753
Genre: Sad
Price: 800
Stock: 12

--- Album Details ---
Album ID: 777
Title: BEST
Artist ID: 687
Genre: Modern
Price: 587
Stock: 22
```

```
===== MUSIC STORE MANAGEMENT SYSTEM =====
1. Add Album
2. Display All Albums
3. Search Album
4. Add Artist
5. Display Artists
6. Add Customer
7. Display Customers
8. Process Sale
9. Exit
Enter your choice: 5

--- Artist Details ---
Artist ID: 544
Name: Arijit Singh
Genre: Romantic

--- Artist Details ---
Artist ID: 789
Name: JINU
Genre: Phonk
```

```
===== MUSIC STORE MANAGEMENT SYSTEM =====
1. Add Album
2. Display All Albums
3. Search Album
4. Add Artist
5. Display Artists
6. Add Customer
7. Display Customers
8. Process Sale
9. Exit
Enter your choice: 3
Enter Album ID to search: 756

--- Album Details ---
Album ID: 756
Title: Thunder
Artist ID: 753
Genre: Sad
Price: 800
Stock: 12
```

```
===== MUSIC STORE MANAGEMENT SYSTEM =====
1. Add Album
2. Display All Albums
3. Search Album
4. Add Artist
5. Display Artists
6. Add Customer
7. Display Customers
8. Process Sale
9. Exit
Enter your choice: 8
Enter Sale ID: 1002
Enter Customer ID: 158
Enter Album ID: 756
Enter Quantity: 5

Sale completed successfully!
Quantity: 5
Total Amount: 4000
Remaining Stock: 7
```

12. Testing

Test Case	Input / Operation	Expected Output	Actual Output
1. Program Start	Run main.exe	Main menu is displayed	Main menu displayed successfully
2. Add Album	Enter valid album details	Album is added successfully	Album added successfully
3. Display Albums	Select option 2	All stored albums are displayed	Album details displayed correctly
4. Search Album	Enter an existing album ID	Matching album details are displayed	Album found successfully
5. Search Invalid ID	Enter a non-existing album ID	Album not found message appears	Album not found
6. Add Artist	Enter valid artist details	Artist is added successfully	Artist added successfully
7. Add Customer	Enter valid customer details	Customer is added successfully	Customer added successfully
8. Process Sale	Enter valid customer ID, album ID, and quantity	Sale is recorded and stock decreases	Sale processed and stock updated

9. Data Persistence	Restart the program	Previously saved records are loaded	Saved records loaded successfully
10. Exit	Select option 9	Data is saved and program terminates	Program exited successfully

13. Challenges and Solutions

1. **Compiler Configuration:** Initially, the C++ compiler was not recognized. This was resolved by installing and configuring GCC through MSYS2.
2. **Compilation Errors:** Missing header files and linker errors occurred during development. These were resolved by checking file names, including the required headers, and compiling all necessary .cpp files together.
3. **Code Organization:** Managing multiple classes in a single file became difficult. The code was organized into separate .h and .cpp files for better readability and maintenance.
4. **File Handling:** Saved records were not always visible in the expected location. This was addressed by checking the project directory and ensuring the program used the correct text files.
5. **Stock and Sales Management:** Ensuring that album stock changed correctly after a sale required testing and debugging. The sale functionality was verified by checking the updated stock and saved transaction records.
6. **Data Persistence:** Records needed to remain available after restarting the program. File loading and saving functions were implemented and tested to preserve data between executions.

These challenges helped improve the program's reliability and strengthened my understanding of C++, object-oriented programming, compilation, and file handling.

14. Conclusion

The Music Store Management System successfully demonstrates how C++ can be used to manage albums, artists, customers, and sales through a menu-driven application. The project helped me understand important Object-Oriented Programming concepts, including classes, objects, encapsulation, inheritance, and polymorphism. I also learned how to organize code into multiple files, handle compilation errors, and use file handling to save and retrieve data. The system simplifies music store operations by maintaining records, processing sales, updating stock, and preserving data between program executions. Overall, this project improved my practical programming, problem-solving, and debugging skills.

GitHub Repository Link

The complete source code for the Music Store Management System is available at:

https://github.com/jitanshuborana123-cloud/Music-Store-Management-System